

Full-Stack Developer Take-Home Assignment

Capital Opportunities Tracker

Thank you for taking the time to complete this assignment. The goal is to understand how you approach a small feature from database design through user interface implementation.

Please prioritize correctness, readable code, and clear decisions over adding extra features or highly polished visuals.

Time expectation

This assignment is designed to be completed within **24–48 hours**. We recommend limiting active development time to approximately **24 hours** and submitting the project no later than 48 hours after receiving it.

You are not expected to build a production-perfect system. If you do not complete everything within the available time, submit what you have and explain the remaining work and intended approach in your README.

Required technology

- Next.js with the App Router
- React
- TypeScript
- Prisma ORM
- PostgreSQL
- Zod for validating untrusted input
- SWR for client-side data queries
- Next.js Server Actions for application data reads and mutations
- NextAuth with a Credentials provider
- **bcrypt** or **bcryptjs** for password hashing
- A styling solution of your choice

You may use Docker to run PostgreSQL, but it is not required.

Project overview

Build a small internal application that allows a team to track funding opportunities submitted by companies.

Each opportunity must contain:

- Company name
- Requested amount
- Currency
- Stage
- Submission date
- Short description

- Assigned reviewer
- Created by
- Created and updated timestamps

The supported stages are:

- DRAFT
- UNDER_REVIEW
- APPROVED
- REJECTED

Functional requirements

1. Opportunities list

Create a page that displays all opportunities in a table or similarly clear layout.

The user must be able to:

- Search by company name
- Filter by stage
- Sort by requested amount
- Sort by submission date
- Paginate through the results
- Open an individual opportunity

Search, filter, sort, and pagination state must be represented in the URL so the current view can be refreshed or shared.

The page must provide appropriate loading, empty, and error states. Filtering, sorting, and pagination must be performed through the server and database rather than by loading the complete dataset into the browser.

2. Opportunity details

Create a dynamic route for viewing one opportunity.

The details page must display all stored fields and provide permitted actions to edit the opportunity, assign a reviewer, change its stage, and add internal comments.

If the requested record does not exist, display an appropriate not-found state.

3. Create and edit forms

Allow users to create and edit opportunities.

Validate input on the server. Client-side validation may also be included for better user feedback.

At minimum, enforce these rules:

- Company name is required and must not contain only whitespace.
- Requested amount must be greater than zero.

- Currency must be a supported three-letter currency code. Supporting **USD**, **EUR**, and **GBP** is sufficient.
- Description is required and must not exceed 500 characters.
- Submission date must be a valid date.
- Stage must be one of the supported values.

Display useful validation messages next to the relevant fields.

4. Stage transitions

Only the following stage changes are allowed:

- **DRAFT** to **UNDER_REVIEW**
- **UNDER_REVIEW** to **APPROVED**
- **UNDER_REVIEW** to **REJECTED**

All other stage changes must be rejected by the server, even if a request is made outside the user interface.

Every successful stage change must create an activity record containing:

- The previous stage
- The new stage
- The user who performed the change
- The date and time of the change

Changing the stage and creating its activity record must succeed or fail as one database operation.

5. Authentication and user roles

Use **NextAuth with a Credentials provider** for authentication. User records must be stored in PostgreSQL through Prisma, and passwords must be hashed with **bcrypt** or **bcryptjs**. Plain-text passwords must never be stored.

NextAuth manages the authentication flow, session, and authentication route. The Credentials provider does not automatically store users or verify their passwords. In the provider's **authorize** function, load the user through Prisma and use **bcrypt.compare** or **bcryptjs.compare** to verify the submitted password against the stored hash.

The application does not require user registration, password reset, social login, or email verification. Seed at least one account for each of these roles:

- **ADMIN**
- **REVIEWER**
- **VIEWER**

Each user has one role. Apply the following role behavior:

Action	Admin	Reviewer	Viewer
View dashboard and opportunities	Yes	Yes	Yes
Create an opportunity	Yes	No	No

Action	Admin	Reviewer	Viewer
Edit opportunity details	Yes	No	No
Assign a reviewer	Yes	No	No
Change the stage	Yes	Yes	No
Add an internal comment	Yes	Yes	No
Archive and restore an opportunity	Yes	No	No

Role checks must be enforced on the server. Hiding a button in the interface is not sufficient protection.

Implement the following authentication behavior:

- Provide a login page and sign-out action.
- Redirect unauthenticated page requests to the login page.
- Preserve the authenticated user's ID and role in the NextAuth session.
- Use secure session cookies through NextAuth. A JWT-based NextAuth session is acceptable.
- Return an unauthenticated or forbidden result when a protected server operation is called without sufficient access.
- Document the seeded login credentials in the README.

Use direct role checks to determine what an authenticated user can do. A separate permissions, capability, or policy system is not required.

Every protected Server Action must perform its own role check. Middleware, protected layouts, disabled controls, and hidden buttons may improve the user experience, but they must not be the only enforcement layer.

The API route or route handler required by NextAuth is an expected exception to the Server Action preference. NextAuth is responsible for handling that authentication endpoint.

Do not accept the acting user ID, role, or **createdBy** value from submitted form data. Derive the acting user's identity and role from the verified NextAuth session. Before a protected write, load the relevant records from PostgreSQL and enforce the role rules and current record state on the server.

When assigning an opportunity, verify that the selected user has the **REVIEWER** role. Role management itself is outside the scope of this assignment.

6. Dashboard

Create a dashboard that displays:

- Total active opportunities
- Number of opportunities in each stage
- Total requested amount grouped by currency
- Five most recently submitted opportunities
- Reviewer workload showing the number of active assigned opportunities per reviewer

The dashboard must be generated from PostgreSQL data. Present the information clearly; charts are optional.

7. Comments and activity history

Authenticated admins and reviewers can add internal comments to an opportunity.

Each comment must show its author and creation time. Empty comments must be rejected, and comment text must not exceed 1,000 characters.

Display a chronological activity timeline on the opportunity details page. It must include:

- Opportunity creation
- Stage changes
- Reviewer assignment changes
- Comments
- Archive and restore actions

Activity records are append-only and must not be editable through the application.

8. Archive and restore

Admins can archive and restore opportunities. Archived opportunities:

- Must be excluded from the default list and dashboard totals
- Must remain stored in PostgreSQL
- Must be accessible through an explicit archived filter
- Cannot be edited or moved to another stage until restored

The server must enforce these restrictions.

9. Data fetching and mutations

Use **SWR** for client-side data queries that power the opportunities list, filters, pagination, dashboard summaries, comments, and activity history.

Use authenticated **Server Actions** as the SWR fetchers for application data. Inputs for search, filters, sorting, and pagination must be validated before they are used in Prisma queries. Do not access Prisma directly from client components.

Use **Next.js Server Actions** for all application mutations, including:

- Creating and editing an opportunity
- Assigning or changing a reviewer
- Changing an opportunity stage
- Adding a comment
- Archiving and restoring an opportunity

Every Server Action must independently:

- Confirm the user's authentication
- Enforce the required user role
- Validate untrusted input with Zod
- Return a useful success or error result
- Refresh or invalidate the relevant SWR data after a successful mutation

The browser must not be treated as the authority for permissions, stage-transition rules, or archive restrictions.

10. Persistence

Use Prisma ORM with PostgreSQL for all data access.

Include:

- A Prisma schema
- A migration that creates the required database structure
- Appropriate relations and database constraints
- A seed script with at least twelve opportunities covering every stage, all roles, multiple reviewers, comments, activity records, and archived records
- Clear local database setup instructions

Do not use raw SQL for ordinary create, read, update, filtering, or sorting operations that Prisma can express.

11. Server Action documentation

Document the Server Actions used for application reads and mutations. For each action, include:

- Purpose
- Required input
- Expected successful result
- Possible validation or authorization failures

A concise Markdown document is sufficient. You do not need to document NextAuth's internal authentication endpoint.

Technical expectations

- Keep secrets and database credentials out of Git.
- Include an `.env.example` containing the required variable names.
- Keep database access and business rules separate from presentation components.
- Keep authentication, role checks, and input validation on trusted server boundaries.
- Avoid using `any` unless there is a clearly documented reason.
- Handle expected failures instead of silently ignoring them.
- Make the interface usable on both desktop and small screens.
- Use semantic HTML and correctly associated form labels.
- Prevent accidental duplicate submissions for create, update, comment, and stage-change actions.
- Provide visible success and failure feedback for mutations.
- Use database transactions where multiple related writes must remain consistent.

File uploads, email notifications, real-time updates, third-party integrations, and production deployment are outside the scope of this assignment.

Optional enhancements

Attempt these only after completing the required functionality:

- Add an audit-log export in CSV format.
- Add a simple chart to the dashboard.
- Add optimistic UI updates with safe error recovery.
- Provide a Docker Compose setup for the application and PostgreSQL.
- Deploy a working demonstration and document any seeded credentials.

Optional enhancements do not compensate for missing core requirements.

Submission requirements

Submit a link to a Git repository containing:

- The complete source code
- The Prisma schema and migration
- Seed data
- Server Action documentation
- An `.env.example` file
- A README with setup and run instructions
- Meaningful Git commits showing how the solution developed

Your README must also briefly explain:

- The main technical decisions you made
- Any requirements you did not complete
- What you would improve with additional time
- Any assumptions you made
- Whether you used AI tools and, if so, how you used them

Do not commit real credentials, generated dependency directories, or database dumps.

Evaluation criteria

Area	Weight
Functional correctness and edge cases	30%
TypeScript and code quality	15%
React, Next.js, Server Actions, and SWR fundamentals	15%
Validation, authentication, and user-role handling	15%
Prisma, PostgreSQL, and data modeling	17%
Accessibility and responsive usability	5%
Documentation and Git history	3%

Follow-up discussion

Be prepared to walk us through your solution, explain the flow from the interface to PostgreSQL, and discuss your security and data-modeling decisions. During the follow-up session, we may ask you to make a small change, adjust a permission rule, or investigate a simple bug in your submission.

We do not expect a production-perfect application. We are interested in how you structure a problem, apply the fundamentals, communicate tradeoffs, and respond to feedback.